

Optimisation de Portefeuille via Algorithme Génétique

Documentation LaTeX

TEAM 137

28 janvier 2025

Table des matières

1	Introduction	2
1.1	Contexte et définition d'un algorithme génétique	2
1.2	Algorithmes génétiques pour l'optimisation de portefeuille	2
2	Architecture Générale du Projet (MVC)	2
2.1	Extrait du <code>main.py</code>	3
3	Présentation de la Classe Portfolio	4
3.1	Rôle et structure	4
3.2	Calcul du coût total, du rendement et de la variance	4
3.3	Fonction de fitness	5
4	Algorithme Génétique : GeneticAlgorithm	5
4.1	Initialisation de la population	5
4.2	Boucle principale (<code>evolve()</code>)	5
5	Sélection, Croisement, et Mutation	6
5.1	Sélection par Tournoi	6
5.2	Croisement (<code>crossover</code>)	6
5.3	Mutation	7
6	Discussion et Améliorations Potentielles	8
7	Conclusion	8

1 Introduction

1.1 Contexte et définition d'un algorithme génétique

Les **algorithmes génétiques (AG)** sont des heuristiques stochastiques inspirées par la théorie de l'évolution et la génétique naturelle. L'idée maîtresse repose sur la sélection naturelle : seuls les individus les plus aptes (en termes de *fitness*) ont de meilleures chances de se reproduire et de propager leurs caractéristiques à la génération suivante.

Un algorithme génétique manipule généralement une *population d'individus*, chacun représentant une solution candidate à un problème donné. Chaque individu est caractérisé par un *génotype* (ou un ensemble de *paramètres*) dont le *phénotype* (ou la solution concrète) est évalué par une *fonction d'adaptation (fitness function)*. Au fil des itérations (ou *générations*), on applique :

- **Sélection** : Choisir des individus plus performants pour la reproduction.
- **Croisement** (*crossover*) : Combiner deux individus (appelés parents) pour créer un ou plusieurs enfants.
- **Mutation** : Introduire de petites perturbations aléatoires dans le génome afin de maintenir la diversité.

Le but est de *faire évoluer* la population vers une zone de l'espace de recherche offrant une bonne ou une excellente solution au problème.

1.2 Algorithmes génétiques pour l'optimisation de portefeuille

Dans le cadre de l'optimisation de portefeuille, en particulier selon l'approche de Markowitz, on souhaite construire un portefeuille d'actifs (actions, obligations, ...) qui maximise le rendement espéré pour un niveau de risque donné (ou, inversement, minimiser le risque pour un rendement souhaité). Ici, nous utilisons une *fonction d'utilité quadratique* ou un *ratio de Sharpe* pour guider la recherche d'un compromis optimal.

Par leur capacité à rechercher un optimum global sans se bloquer trop facilement dans des minima locaux, les algorithmes génétiques se prêtent bien à ce type de problème multivariable et potentiellement non-convexe.

Dans la suite, nous décrivons la structure de notre projet, ses composants principaux (*controller*, *model*, *view*), et les différentes étapes de l'implémentation de l'algorithme génétique (initialisation, sélection, croisement, mutation, ...).

2 Architecture Générale du Projet (MVC)

Le projet suit un patron d'architecture **MVC** (*Model-View-Controller*) :

- **Model** : Contient la logique métier, les classes `GeneticAlgorithm`, `Portfolio`, `Stock`, etc.
- **View** : Gère l'affichage, ici principalement `PortfolioVisualizer` et `GAPParameterView`.
- **Controller** : Lance et gère le flux général du programme. On y retrouve par exemple `main.py`.

L'utilisateur sélectionne différents paramètres (taille de population, type de fonction fitness, méthode de sélection, etc.) via l'interface (*view*). Les données (*model*) sont traitées et le contrôleur (*controller*) orchestre l'ensemble.

2.1 Extrait du main.py

Le fichier `controller/main.py` illustre le point d'entrée du programme (voir extrait ci-dessous). On y initialise le chargement des données, crée la fenêtre de saisie (paramètres de l'AG), et enfin lance l'algorithme génétique.

Listing 1 – Extrait de `controller/main.py`

```
def main():
    file_path = 'data/Cac40_Prices_2000_to_Today.xlsx'
    data_loader = DataLoader(file_path)

    try:
        stocks, cov_matrix = data_loader.get_data()
    except Exception as e:
        print(f"An error occurred: {e}")
        return

    # Cration de la fenetre pour la slection des paramtres
    root = tk.Tk()
    root.withdraw()

    param_view = GAPParameterView(master=root)
    root.wait_window(param_view)

    if not param_view.parameters_confirmed:
        print("No parameters selected. Exiting.")
        return

    params = param_view.get_parameters()

    try:
        ga = GeneticAlgorithm(
            stocks,
            cov_matrix,
            population_size=params["population_size"],
            is_short_available=params["is_short_available"],
            fitness_function=params["fitness_function"],
            crossover_function=params["crossover_function"],
            mutation_function=params["mutation_function"],
            selection_method=params["selection_method"],
            risk_aversion=params["risk_aversion"],
            budget=params["budget"],
            max_generations=params["max_generations"]
        )

        best_portfolio = ga.evolve(fitness_threshold=15)

        print("Best portfolio:")
        for stock, shares in zip(best_portfolio.stocks, best_portfolio.shares):
            print(f"{stock.name}: {shares} shares")
        print(f"Expected Return: {best_portfolio.calculate_expected_return()*100}%")
        print(f"Variance: {best_portfolio.get_volatility_percentage():.2f}")

        time.sleep(1000)

    except Exception as e:
        print(f"Erreur inconnue:\n{e}")
```

3 Présentation de la Classe Portfolio

3.1 Rôle et structure

La classe `Portfolio` (fichier `model/portfolio.py`) représente un portefeuille d’actions. Son but est de regrouper :

- un vecteur `shares` contenant la quantité (positive ou négative selon si le court-circuitage *short selling* est autorisé) pour chaque action,
- la liste des actions sous-jacentes (`stocks`),
- la matrice de covariance (`cov_matrix`) pour calculer le risque,
- plusieurs méthodes de croisement et de mutation (Simulated Binary Crossover, BLX-Alpha, etc.).

3.2 Calcul du coût total, du rendement et de la variance

- **Coût total** : la somme des nombres de parts multipliées par les prix.

$$\text{Investment} = \sum_{i=1}^n (\text{shares}_i \times p_i)$$

- **Rendement attendu** : on définit des *poids* (fractions relatives de la valeur totale). Si w_i est le poids de l’actif i , le rendement espéré est

$$\sum_{i=1}^n w_i \mu_i$$

où μ_i est le rendement attendu de l’action i .

- **Variance** : à partir des poids, la variance du portefeuille est

$$w^\top \Sigma w \times (\text{Investment})^2$$

avec Σ la matrice de covariance.

Listing 2 – Extrait de la classe `Portfolio`

```
def calculate_total_investment(self):
    prices = np.array([stock.price for stock in self.stocks])
    shares = np.abs(self.shares)
    return np.dot(shares, prices)

def calculate_expected_return(self):
    expected_returns = np.array([stock.expected_return for stock in self.stocks])
    prices = np.array([stock.price for stock in self.stocks])
    total_investment = np.dot(prices, self.shares)
    weights = (prices * self.shares) / total_investment
    return np.dot(weights, expected_returns)

def calculate_variance(self):
    prices = np.array([stock.price for stock in self.stocks])
    investment = prices * self.shares
    weights = investment / self.total_investment
    return np.dot(weights.T, np.dot(self.cov_matrix, weights)) * self.total_investment
    **2
```

3.3 Fonction de fitness

Deux approches de la fonction de fitness ont été implémentées :

1. **Utilité quadratique :**

$$U = R - \frac{\lambda}{2}\sigma^2$$

où R est le rendement par unité (ou par euro investi), σ^2 la variance par unité, et λ l'aversion au risque.

2. **Ratio de Sharpe (simplifié) :**

$$\text{Sharpe} = \frac{\sqrt{R}}{\sigma^2}$$

Ici, l'implémentation simplifie un peu l'interprétation (pas de taux sans risque explicite), mais l'idée reste de valoriser le rapport rendement / risque.

4 Algorithme Génétique : GeneticAlgorithm

La classe `GeneticAlgorithm` centralise l'ensemble de la boucle d'évolution. Elle contient :

- **Initialisation** de la population,
- **Boucle d'évolution**, avec sélection, croisement, mutation,
- **Critères d'arrêt**, par exemple un nombre maximal de générations ou un objectif de fitness,
- **Contrôle de la stagnation** (peut être implémenté pour arrêter si aucune amélioration n'est constatée).

4.1 Initialisation de la population

Soit B le budget et n le nombre d'actions. Une façon de procéder est de générer des quantités aléatoires pour chaque action, puis de rescaler afin de respecter (ou presque) le budget total.

- On peut tirer $s_i \sim \mathcal{U}(0, 1)$ pour chaque action,
- Multiplier s_i par le budget normalisé (ex. B/p_i) pour estimer un nombre de parts,
- Appliquer un *floor* ou un *scaling* pour tenir compte des contraintes finales.

4.2 Boucle principale (evolve())

Durant chaque génération :

1. On calcule la fitness pour tous les individus de la population.
2. On sélectionne des couples de parents (tournoi, etc.).
3. On effectue le croisement, formant des enfants.
4. On applique la mutation sur ces enfants.
5. On remplace éventuellement certains individus de la population (selon l'élitisme, etc.).
6. On vérifie le critère d'arrêt (génération max atteinte, fitness suffisamment élevée).

Listing 3 – Extrait possible de la boucle d'évolution (simplifiée)

```
def evolve(self, fitness_threshold=None):
    for generation in range(self.max_generations):
        # 1. Calcul de la fitness
        for portfolio in self.population:
```

```

        portfolio.calculate_fitness()

# 2. Trouver le meilleur de la gnration
best_individual = max(self.population, key=lambda p: p.fitness)

if fitness_threshold is not None:
    if best_individual.fitness >= fitness_threshold:
        print(f"Threshold reached at generation {generation}")
        return best_individual

# 3. Cration de la nouvelle population via tournoi, croisement, mutation
new_population = []

while len(new_population) < self.population_size:
    parent1, parent2 = Portfolio.selection(self.population, self.
        selection_method)
    child = parent1 + parent2 # croisement
    child = ~child # mutation
    new_population.append(child)

# 4. litisme (optionnel) : conserver le meilleur
new_population.append(best_individual)

# 5. Mise jour de la population
self.population = new_population

# Retourne le meilleur si la boucle se termine
return max(self.population, key=lambda p: p.fitness)

```

5 Sélection, Croisement, et Mutation

5.1 Sélection par Tournoi

Le *tournoi* consiste à tirer aléatoirement k individus dans la population et à sélectionner le meilleur parmi ce sous-ensemble. Dans le code :

Listing 4 – Sélection par tournoi dans Portfolio

```

@staticmethod
def tournament_selection(population, k=3):
    selected = []
    for _ in range(2):
        individuals = np.random.choice(population, k)
        best = max(individuals, key=lambda ind: ind.fitness)
        selected.append(best)
    return selected

```

5.2 Croisement (crossover)

Trois méthodes principales de croisement ont été implémentées :

1. **Simulated Binary Crossover (SBX)** : Contrôle la dispersion via l'indice de distribution

η .

$$\beta = \begin{cases} (2u)^{1/(\eta+1)} & \text{si } u \leq 0.5 \\ \left(\frac{1}{2(1-u)}\right)^{1/(\eta+1)} & \text{sinon} \end{cases}$$

L'enfant est une combinaison pondérée des valeurs parentales :

$$\text{child_share} = 0.5 [(1 + \beta)p_1 + (1 - \beta)p_2].$$

2. **Blend Crossover (BLX- α)** : On définit un intervalle autour des valeurs parentales et on tire une valeur aléatoire dans cet intervalle.
3. **Arithmetic Crossover** : On prend une combinaison linéaire simple, par exemple $r \times p_1 + (1 - r) \times p_2$.

Listing 5 – Simulated Binary Crossover (SBX)

```
def simulated_binary_crossover(self, other):
    child_shares = []
    for i in range(len(self.shares)):
        u = np.random.rand()
        if u <= 0.5:
            beta = (2 * u) ** (1 / (Portfolio.eta + 1))
        else:
            beta = (1 / (2 * (1 - u))) ** (1 / (Portfolio.eta + 1))
        child_share = 0.5 * ((1 + beta) * self.shares[i] + (1 - beta) * other.shares[i])
        child_shares.append(child_share)
    if not self.is_short_available:
        child_shares = np.maximum(child_shares, 0)
    child_portfolio = Portfolio(child_shares, ...)
    child_portfolio.adjust_shares_to_budget()
    child_portfolio.calculate_fitness()
    return child_portfolio
```

5.3 Mutation

Trois variantes de mutation :

1. **Mutation Gaussienne** : On perturbe la valeur par un bruit normal de moyenne 0 et d'écart-type $\sigma \times |\text{valeur}|$.

$$\text{valeur} \leftarrow \text{valeur} + \mathcal{N}(0, \sigma |\text{valeur}|)$$

2. **Polynomial Mutation** : Contrôle via η_m (similaire à SBX).
3. **Uniform Mutation** : Remplace la valeur par une valeur aléatoire uniforme dans une plage autour de l'ancienne valeur.

Listing 6 – Mutation gaussienne

```
def gaussian_mutation(self):
    mutated_shares = self.shares.copy()
    for i in range(len(mutated_shares)):
        if np.random.rand() < Portfolio.mutation_rate:
            mutated_shares[i] += np.random.normal(0, Portfolio.sigma * abs(
                mutated_shares[i]))
    if not self.is_short_available:
        mutated_shares = np.maximum(mutated_shares, 0)
    mutated_portfolio = Portfolio(mutated_shares, ...)
```

```
mutated_portfolio.adjust_shares_to_budget()
mutated_portfolio.calculate_fitness()
return mutated_portfolio
```

6 Discussion et Améliorations Potentielles

Bien que l'implémentation soit fonctionnelle et intègre les principales composantes d'un algorithme génétique, il existe plusieurs pistes d'améliorations :

- **Gestion plus stricte du budget** : Actuellement, après à chaque nouvelle génération on re scale les portefeuilles pour respecter le budget. On pourrait envisager des stratégies comme ajouter une pénalité pour le dépassement du budget mais on s'est rendu compte que cela rajoutait une contrainte inutile puisqu'il est possible de scale le portefeuille pour fit le budget. On définitive cela rend juste la convergence moins efficace.
- **Introduction de contraintes supplémentaires** : Dans un contexte réel, on pourrait limiter la part d'un même actif, interdire certains secteurs, ou ajouter des contraintes de liquidité.
- **Contrôle de la stagnation** : On pourrait envisager d'utiliser des formules type reinforcement learning pour une meilleure gestion de la stagnation (càd en utilisant un facteur qui mesure à quel point la nouvelle génération fait progresser la moyenne des générations et accélère / réduit la diversité en fonction.
- **Hybridation avec d'autres heuristiques** : On peut combiner l'AG avec des méthodes de gradient local sur certaines parties (approches hybrides), ou introduire des opérations de *swarm intelligence*.
- **Paramétrage automatique** : Ajuster dynamiquement (au fil des générations) les taux de mutation, η , etc., selon la diversité observée ou la convergence de la population.
- **Visualisation approfondie** : Le PortfolioVisualizer est un bon début, mais on peut imaginer des graphiques pour la répartition sectorielle, etc.

7 Conclusion

Cet algorithme génétique pour l'optimisation de portefeuille démontre la faisabilité de l'approche sur un cas concret (CAC40). En jouant sur les différents paramètres (choix de la fonction de fitness, taux de mutation, taille de la population, etc.), l'utilisateur peut explorer divers compromis risque/rendement.

Cette implémentation *MVC* clarifie également la structure (modèle, vue, contrôleur). Enfin, les pistes d'améliorations mettent en évidence le caractère modulable de la solution, facilement extensible à d'autres types de croisement, de mutation, ou de règles de sélection afin de mieux refléter les conditions réelles des marchés.

Fin du document.